

Project Report
On
“AUTOMATED LIBRARY ASSISTANT”
Completed at
LYALLPUR KHALSA COLLEGE TECHNICAL CAMPUS



Submitted for the partial fulfilment for the award of the degree of
Bachelor of Technology
IN
(Data Science)

Submitted By:
SARBJOT SINGH & MUNISH KUMAR
University Roll No: 2233367 & 2233361

SUBMITTED TO:
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
LYALLPUR KHALSA COLLEGE TECHNICAL CAMPUS
JALANDHAR
Session 2025-26

“AUTOMATED LIBRARY ASSISTANT”
COMPLETED AT
LYALLPUR KHALSA COLLEGE TECHNICAL CAMPUS
JALANDHAR

Submitted for the partial fulfillment for the award degree of
Bachelor of Technology
IN
(Data Science)



Submitted By:
SARBJOT SINGH & MUNISH KUMAR
University Roll No: **2233367 & 2233361**

SUBMITTED TO:
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
LYALLPUR KHALSA COLLEGE TECHNICAL CAMPUS
JALANDHAR

TABLE OF CONTENTS

S. No.	Topic	PAGE NO.
	Declaration Abstract Acknowledgement List of Figures and tables	i ii iii iv
1	CHAPTER – 1 INTRODUCTION 1.1 Introduction 1.2 Problem Statement 1.3 Project Motivation	1-2
2	CHAPTER – 2 LITERATURE REVIEW	3-4
3	CHAPTER – 3 TECHNOLOGY USED 3.1 Backend Technologies 3.2 Frontend Technologies 3.3 Machine Learning & AI Libraries 3.4 IoT & Hardware Integration 3.5 Payment Gateway API	5-12
4	CHAPTER – 4 NEED, SCOPE, AND OBJECTIVE OF THE STUDY 4.1 Need for the Proposed System 4.2 Scope of the Project 4.3 Objectives	13-15

5	CHAPTER – 5 PROJECT WORK UNDERTAKEN 5.1 Feasibility Study 5.2 System Architecture 5.3 Database Design (ER Diagram) 5.4 Secure IoT Transaction Workflow 5.5 Fine Calculation & Payment Methodology 5.6 Security & Password Management 5.7 Machine Learning Model Development 5.8 AI Chatbot Methodology	16-27
6	CHAPTER – 6 RESULT AND DISCUSSION 6.1 Hardware Implementation 6.2 Software Implementation 6.3 IoT Kiosk Interface and User Feedback 6.4 User Dashboard 6.5 Loan Status & Risk Prediction Tiles 6.6 Fine Payment System (PayPal) 6.7 AI Chatbot 6.8 Admin Dashboard 6.9 Transaction Authorization Flow & E-Receipt	28-38
7	CHAPTER – 7 CONCLUSION AND FUTURE SCOPE 7.1 Conclusion 7.2 Future Scope 7.3 Recommendations of the Study	39-40
BIBLIOGRAPHY		41-42
Appendix A. Glossary of Terms B. Abbreviations C. User Survey Questionnaire D. Sample E-Receipt Proforma		43-45

CANDIDATE'S DECLARATION

We here by certify that the work which is being presented in the project report entitled “Automated Library Assistant” by “Sarbjot Singh” & “Munish Kumar” in partial fulfillment of requirements for the award of degree of B.TECH (DS) submitted in the Department of CSE at “LYALLPUR KHALSA COLLEGE TECHNICAL CAMPUS , JALANDHAR” under “I.K.G. PUNJAB TECHNICAL UNIVERSITY, JALANDHAR” is an authentic record of my own work carried out during a period from July 2025 to Dec 2025 under the supervision of “ER. Vandana”.

Sarbjot Singh _____

Munish Kumar _____

This is to certify that the above statement made by the candidate's is correct to the best of my/our knowledge.

Signature of Supervisor.....

The viva –voce of “Sarbjot Singh & Munish Kumar” is accepted.

Signature of External Examiner

Signature of HOD.....

ABSTRACT

This project, "Automated Library Assistant," solves the inefficiencies of traditional library management by developing an integrated system using IoT, web technologies, and machine learning. The system is built on a Python-Flask backend and MySQL database, providing a modern web portal for users and administrators.

A key innovation is the secure IoT kiosk, which uses an Arduino and RFID reader for physical user authentication before any transaction. The system leverages machine learning to provide intelligent features: a hybrid book recommendation engine (combining content-based and collaborative filtering) and a predictive model to identify loans at high risk of being late.

Additional features include a 24/7 hybrid AI chatbot powered by Google Gemini , an automated fine management system integrated with the PayPal API , and a comprehensive admin dashboard. The result is a robust, scalable prototype that automates manual tasks, enhances security, and creates an efficient, data-driven experience for the modern academic library .

ACKNOWLEDGEMENT

First and foremost, we would like to express our deepest gratitude towards our project supervisor Er. Vandana, for his invaluable guidance, insightful feedback, and unwavering support throughout the duration of this project. His expertise and encouragement have been instrumental in shaping this research and helps in the successful completion of this project on Automated Library Assistant.

We are also thankful to Lyallpur Khalsa College, Technical Campus for providing the necessary resources and facilities that enabled us to conduct our experiments and analyses effectively.

Place: LKCTC, Jalandhar
Date:

Sarbjot Singh

(2233367)

Munish Kumar

(2233361)

Internal Examiner.....

HOD (CSE).....

External Examiner.....

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Fig 3.1.1	Python Logo	5
Fig 3.1.2	Flask Framework Logo	6
Fig 3.1.3	MySQL Database Logo	6
Fig 3.2.1	HTML5 Logo	7
Fig 3.2.2	Tailwind CSS Logo	7
Fig 3.2.3	JavaScript Logo	8
Fig 3.3.1	Jupyter Notebook Logo	8
Fig 3.3.3	Pandas Library Logo	9
Fig 3.3.4	Hugging Face Logo	9
Fig 3.3.5	Google Gemini API Logo	10
Fig 3.4.1	Arduino Logo	10
Fig 3.4.2	PySerial Library Logo	11
Fig 3.4.3	I2C 16x2 LCD Display	11
Fig 3.5.1	PayPal Logo	12
Fig 5.1	System Architecture Diagram	17
Fig 5.2	Database Entity-Relationship (ER) Diagram	20
Fig 5.3	Secure IoT Transaction Workflow	20
Fig 5.4	Content-Based Filtering Workflow	25
Fig 5.5	Collaborative Filtering Workflow	25
Fig 5.6	AI Chatbot Methodology Flow	27
Fig 6.1	Hardware Implementation of IoT Kiosk	28
Fig 6.2	Login Page User Interface (Dark Mode)	29
Fig 6.3	Reset Password Page User Interface	29
Fig 6.4	IoT Kiosk LCD Feedback (Successful Auth)	30
Fig 6.5	IoT Kiosk LCD Feedback (Access Denied)	30
Fig 6.6	User Dashboard: "My Books" Tab	31
Fig 6.7	User Dashboard: "Recommendations" Tab	31
Fig 6.8	User Dashboard: "Settings" Tab	32
Fig 6.9	Loan Status Tile: "Low Risk"	32
Fig 6.10	Loan Status Tile: "High Risk"	32
Fig 6.11	Loan Status Tile: "Overdue"	33
Fig 6.12	Loan Status Tile: "Fine Paid"	33
Fig 6.13	Fine Payment Alert UI with PayPal Button	33
Fig 6.14	PayPal Sandbox Payment Interface	34
Fig 6.15	AI Chatbot: Database-Specific Query	34
Fig 6.16	AI Chatbot: General Knowledge (Gemini) Query	35
Fig 6.17	Admin Dashboard: RFID Verification Screen	35
Fig 6.18	Admin Dashboard: "Overview" Tab	36
Fig 6.19	Admin Dashboard: "User Management" Tab	36
Fig 6.20	Admin Dashboard: "Book Management" Tab	37
Fig 6.21	Transaction Flow: Step 1 (Authorize)	37
Fig 6.22	Transaction Flow: Step 2 (Identify Book)	38
Fig 6.23	Transaction Flow: Success and E-Receipt	38

LIST OF TABLES

Table No.	Table Title	Page No.
Table 2.1	Literature Review of Existing Systems	3-4
Table 5.1	Database Schema: 'users' Table	18
Table 5.2	Database Schema: 'books' Table	18
Table 5.3	Database Schema: 'transactions' Table	19
Table 5.4	Database Schema: 'payment_log' Table	19
Table 5.5	AI Chatbot Rule-Based Intents	26
Table A.1	Abbreviations	44

CHAPTER – 1

INTRODUCTION

1.1 Introduction

Libraries have long been revered as fundamental pillars of education, research, and societal progress. In a university or college environment, the library serves as the intellectual heart of the campus, providing students and faculty with the essential resources needed for scholarly pursuits. In the modern digital age, the role of the library has evolved beyond a mere repository of books to become a dynamic hub for information access, collaborative learning, and knowledge creation. However, to remain effective and relevant, these institutions must adapt and overcome the persistent inefficiencies inherent in traditional management practices.

The efficiency of many academic libraries today is still hampered by outdated, manual checkout and return processes. These legacy systems, often involving manual data entry or basic barcode scanning, are not only labor-intensive for library staff but also frustratingly time-consuming for students. Queues at the circulation desk, especially during peak hours, detract from the learning environment. Furthermore, these manual methods are highly susceptible to human error, leading to a cascade of problems including inaccurate inventory records, misplaced books, and inefficient tracking of borrowed items. This results in a frustrating experience for users who may be unable to find a book that the system claims is available.

Perhaps the most significant drawback of these traditional systems is their inability to capture and leverage valuable data. They function as simple record-keepers, failing to provide actionable insights into user behavior, resource popularity, or collection deficiencies. Librarians are left to make crucial decisions about purchasing new materials and allocating budgets based on intuition rather than empirical evidence. This "data-poor" environment prevents the library from proactively serving its community and optimizing its collection to meet the evolving academic needs of its students.

1.2 Problem Statement

Based on the challenges in traditional library management, the core problems this project aims to solve are:

1. **Inefficiency:** Manual check-out and check-in processes are slow, require significant staff intervention, and lead to bottlenecks at the circulation desk.

2. **Insecurity:** Basic systems lack robust user authentication, making it difficult to securely link a transaction to a specific, physically present user.
3. **Poor User Experience:** Users have no centralized digital platform to view their loans, discover new books, pay fines, or get help, leading to a disconnected and frustrating experience.
4. **Lack of Data-Driven Insights:** These systems do not leverage modern data science to provide value, such as predicting overdue books or recommending relevant content.
5. **Reactive Fine Management:** Fine systems are often punitive and manual, requiring users to pay in person. There is no automated or recurring system to manage this process.

This project, "Automated Library Assistant," proposes a modern, integrated solution to solve these problems by combining IoT, web technologies, and machine learning.

1.3 Project Motivation

Our motivation is to address these critical shortcomings by proposing a paradigm shift from a reactive, manual system to a proactive, intelligent one. We combine the tangible security of Internet of Things (IoT) hardware with the power and accessibility of a data-driven web application. The system is architected to automate tedious administrative tasks, provide a seamless and engaging experience for users, and unlock the potential of data analytics to create a smarter library ecosystem.

The core of the system's innovation lies in its hybrid approach. A physical IoT kiosk, built with an Arduino and an RFID reader, serves as a secure point of authentication. This ensures that a user must be physically present to perform a critical transaction like borrowing or returning a book, drastically reducing the potential for unauthorized actions. This physical "handshake" is then seamlessly linked to a dynamic web portal, which acts as the user's central hub for all library interactions.

From this portal, accessible on any device, a student can view a personalized dashboard of their current loans and borrowing history, search the entire catalog in real-time, and interact with intelligent features powered by machine learning. A sophisticated hybrid book recommendation system helps users discover new and relevant materials. A predictive model will analyze borrowing data to identify loans at high risk of being late, allowing the system to send proactive alerts. Finally, an intelligent AI chatbot will serve as a 24/7 virtual assistant, instantly answering common queries.

By integrating these technologies, the "Automated Library Assistant" aims not only to solve long-standing operational challenges but also to redefine the modern library experience, making it more efficient, engaging, and intelligent.

CHAPTER - 2

LITERATURE REVIEW

A review of existing literature was conducted to understand the current landscape of smart library systems, recommendation engines, and predictive modeling in an academic context. While many studies have focused on individual components, a significant gap exists in integrating all these features into a single, cohesive system with secure IoT authentication.

Title & Source	Year	ML/Technology Used	Limitations	How Our System is Better
"RFID-Based Smart Library Management System" (Kaur & Kaur)	2018	RFID	Focuses primarily on inventory and check-out, lacking a user-facing web portal and data-driven ML features.	Integrates RFID as a <i>secure authentication layer</i> for a full-featured web portal with ML-driven insights.
"A Web-based book recommendation system using collaborative filtering" (Sarmah & Bhuyan)	2015	Collaborative Filtering	Relies only on collaborative filtering, which suffers from the "cold start" problem (can't recommend new books).	Uses a <i>hybrid</i> model (Content-based + Collaborative) to provide relevant recommendations for all books and users.
"A new approach for library management system using RFID technology" (Gudadhe et al.)	2010	RFID	An early system focused on replacing barcodes. Lacks modern web integration or any ML features.	Uses RFID not just for tracking, but as a secure authorization method for the web portal.
"An IoT-Based Framework for a Modern Smart Library" (Patel & Shah)	2019	IoT (General Framework)	Proposes a high-level framework but doesn't integrate secure transactions with a user-facing ML portal.	Implements a <i>specific</i> IoT-to-web workflow (RFID scan unlocks web UI) tied to the user's account.

"Implementing a Hybrid Book Recommendation System..." (O'Connor & Brien)	2017	Hybrid Recommendation System	Focuses <i>only</i> on the recommendation feature. Not part of a larger, integrated system.	Our recommender is one of several integrated ML features (late prediction, chatbot) in a complete management system.
"Predicting Overdue Books in Academic Libraries..." (Chen & Li)	2020	ML Classification	A standalone analytical study. The prediction is not integrated into a live dashboard for users.	Our system <i>operationalizes</i> this concept, displaying the "Late Risk" prediction to users in real-time on their dashboard.
"Design... of a Python-Flask Based Web Application for University Library..." (Ali & Kumar)	2021	Python, Flask	Provides a good web app but lacks secure IoT integration for transactions and advanced ML features.	Builds on this same robust Flask stack but adds the secure IoT kiosk, ML-driven predictions, and an AI chatbot.
"A Chatbot for Library Services: A Case Study..." (Nurcan & Halil)	2019	Chatbot (Rule-based)	Limited to predefined rules (e.g., library hours). It fails on general knowledge or complex questions.	Implements a <i>hybrid</i> chatbot that uses rules for database queries (e.g., "Is book available?") and falls back to Google Gemini for general queries.
"RFID and Web-based an Integrated Library Management System" (Sonawane & Pardeshi)	2016	RFID, Web Integration	A good early integration, but the web portal is a basic management system, not a modern, interactive SPA.	Our system is a modern Single Page Application (SPA) with a dynamic UI, secure auth polling, and integrated AI features.
"Data mining and machine learning in the context of library..." (Zhao & Sung)	2014	Data Mining (General)	A theoretical paper discussing the <i>potential</i> of ML in libraries. It does not propose a specific system.	Our project is a practical <i>implementation</i> of these ideas, integrating multiple ML models into a working product.

Table 2.1

CHAPTER – 3

TECHNOLOGY USED

This project leverages a modern, full-stack architecture. The technologies are chosen to be open-source, powerful, and well-suited for a data-intensive web application with IoT integration.

3.1 Backend Technologies

3.1.1 Python:

Python is a high-level, dynamic programming language used for the entire backend. Python 3.7+ is used due to its extensive open-source libraries, simple syntax, and powerful data processing capabilities. It serves as the foundation for both the web server and all machine learning models. In this project, Python acts as the "glue" that connects the web interface, the database, the machine learning models, and the IoT hardware into a single, functioning application



Fig. 3.1.1

3.1.2 Flask:

Flask is a lightweight and flexible micro web framework for Python. It is used to build the backend server and the RESTful API for the "Automated Library Assistant." Flask handles all web page routing, processes HTTP requests from the web frontend and the IoT kiosk, and communicates with the MySQL database to manage all data operations.

Specifically, we use Flask for:

- **Routing:** Defining all API endpoints (e.g., /api/login, /api/user/dashboard, /api/book/borrow).
- **Session Management:** Securely managing user login state using server-side sessions. This ensures that only authenticated users can access their dashboard or perform transactions.

- **Request Handling:** Processing incoming JSON data from the frontend JavaScript (`request.get_json()`) for all user actions.
- **Response Generation:** Formatting and sending JSON responses (`jsonify()`) back to the frontend to confirm success or provide specific error messages.



Fig 3.1.2

3.1.3 MySQL:

A powerful, open-source relational database management system (RDBMS) used to store all persistent data. This includes the users table (for credentials, roles, and fine status), the books table (for the complete inventory and image URLs), the transactions table (for all loan records), and the payment_log table (for tracking payment gateway attempts). The mysql-connector-python library is used within Flask to execute raw SQL queries for all database operations.



Fig 3.1.3

3.2 Frontend Technologies

3.2.1 HTML (HyperText Markup Language) :

HTML is the standard markup language used to create the fundamental structure and skeleton of the web portal. It defines all the elements of the user interface, such as forms, buttons, text, and containers for the dashboard and all other pages. This project uses a single-page application (SPA) style, where one

index.html file contains all the "pages" (Login, Dashboard, Admin, etc.) as different div elements, and JavaScript is used to show or hide them.



Fig 3.2.1

3.2.2 Tailwind CSS:

A utility-first CSS framework used for styling the web application. Instead of writing custom CSS, Tailwind provides low-level utility classes (e.g., bg-blue-600, text-lg, dark:bg-gray-800) to build modern, responsive designs directly in the HTML markup. This enabled the rapid development of the clean, card-based UI and the implementation of the automatic, system-based dark/light theme system.



Fig 3.2.2

3.2.3 JavaScript:

JavaScript is the scripting language that makes the web portal dynamic and interactive. It is essential for handling all client-side logic, such as:

- **Event Handling:** Listening for user events (e.g., button clicks, form submissions).
- **API Communication:** Using the fetch() function to communicate with the Flask API, sending and receiving data as JSON without reloading the page.
- **DOM Manipulation:** Dynamically rendering the user's book list, recommendations, and error messages by changing the HTML content.

- **Real-time Polling:** Powering the startAuthPolling function, which repeatedly checks the /api/user/status endpoint every 2 seconds to wait for the physical RFID scan.
- **Payment Integration:** Rendering the PayPal button and handling the successful payment callback.



Fig 3.2.3

3.3 Machine Learning & AI Libraries

3.3.1 Jupyter Notebook:

The Jupyter Notebook is an open-source web application used to create and share documents that contain live code, equations, visualizations, and text. It was the primary tool used for the research and development phase of the machine learning models. It allowed for rapid prototyping, data cleaning, model training, and performance evaluation before the final models were saved (`joblib.dump()`) for use in the live application.



Fig 3.3.1

3.3.2 Scikit-learn:

A popular Python library used to build the project's intelligent features. Specifically, we used:

- **NearestNeighbors:** This model is the core of the collaborative filtering part of the recommendation system. It is trained on a pivot table of user ratings to find users with similar tastes.
- **RandomForestClassifier:** This model was trained on historical transaction data to predict the probability of a book being returned late. It uses features like `day_of_week_borrowed` to make its prediction.

3.3.3 Pandas:

A fundamental library for data manipulation and analysis, Pandas was used for creating and managing the DataFrames required to clean, pre-process, and structure the data before training the machine learning models. It was essential for creating the user-item matrix for the collaborative filtering model.



Fig 3.3.3

3.3.4 Sentence-Transformers (Hugging Face):

This library provides an easy method to compute dense vector embeddings (numerical representations) for text. We used the pre-trained all-MiniLM-L6-v2 model to convert all book titles and authors into 384-dimension numerical vectors. These vectors are then compared using `cosine_similarity` to find "content-based" recommendations (i.e., books that are semantically similar).



Fig 3.3.4

3.3.5 Google Gemini:

The project uses the google-generativeai library to interact with the Google Gemini API. This is leveraged in the hybrid chatbot. If a user's query does not match a predefined rule (like checking book availability), the query is securely passed to the Gemini model, which formulates a helpful, conversational response for general knowledge questions.



Fig 3.3.5

3.4 IoT & Hardware Integration Technologies

3.4.1 Arduino:

An open-source electronics platform based on easy-to-use hardware and software. An Arduino Uno board, programmed in C++, is used with an MFRC522 RFID reader to create the physical authentication kiosk. This component reads the user's ID card and sends the UID over the USB serial port.



Fig 3.4.1

3.4.2 Python Kiosk Bridge (PySerial):

The "brain" of the IoT kiosk, this is a separate Python script (`kiosk_bridge.py`) that runs on the kiosk computer. It uses the **PySerial** library, which allows Python to read and write data to hardware serial ports. This script:

- Listens for a UID from the Arduino (via USB).
- Sends that UID to the Flask server's `/api/authorize` endpoint using the requests library.
- Receives a JSON response from the server (e.g., `{"status": "success", "user_name": "Sarbjot"}`).
- Writes a new command *back* to the Arduino (e.g., `SUCCESS:Sarbjot`) to be displayed on the LCD.



Fig 3.4.2

3.4.3 LiquidCrystal_I2C (LCD Display):

This is an Arduino library used to control a 16x2 character LCD display over the I2C protocol. This protocol is highly efficient, requiring only two data pins (SDA and SCL) to control the screen. This component is used to give immediate physical feedback, greeting users by name on success or showing a clear error message (e.g., "RFID Not Found") on failure. A custom character was also designed to display a smiley face.

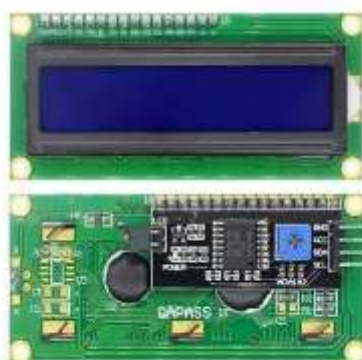


Fig 3.4.3

3.5 Payment Gateway API

3.5.1 PayPal API

The system uses the PayPal REST API to handle fine payments. It is implemented in a secure, server-side flow:

1. **Create Order:** When a user clicks "Pay," the frontend calls the Flask backend. The server calculates the total fine (in USD) and makes a secure, server-to-server call to PayPal to create a payment order. It returns the orderID to the frontend.
2. **On-Approve:** The user approves the payment in the PayPal sandbox popup.
3. **Capture Order:** The frontend tells the Flask backend to "capture" the payment using the orderID. The backend makes a second secure call to PayPal.
4. **Confirmation:** On a successful capture, the backend updates the database, clearing the user's fine and setting the `fine_paid_until` date.



Fig 3.5.1

CHAPTER - 4

NEED, SCOPE AND OBJECTIVES OF THE STUDY

4.1 Need for the Proposed System

The proposed "Automated Library Assistant" is necessary to solve the tangible problems faced by modern academic libraries.

- **Need for Efficiency:** Manual processes are a bottleneck. This system automates check-outs, returns, fine payments, and user support, freeing up library staff to focus on more valuable tasks like research assistance and collection curation.
- **Need for Security:** Basic systems are insecure. By requiring a physical RFID scan for a time-limited digital token, our system ensures that a user is physically present for transactions, preventing unauthorized activity.
- **Need for Engagement:** Libraries are no longer just repositories; they are centers for discovery. A simple catalog is not enough. Users need intelligent tools like recommendation engines and AI chatbots to help them navigate resources and discover new information.
- **Need for Data-Driven Policy:** Libraries make expensive decisions about purchasing books and managing resources. Our system provides the data to make these decisions intelligently, based on user behavior (borrowing patterns) and predictive models (late return risk).
- **Need for Modernization:** Today's students expect a seamless, digital-first experience. A modern, dark-mode-enabled web portal that connects to a physical kiosk and a payment gateway meets this expectation, making the library feel relevant and user-friendly.

4.2 Scope of the Project

The scope of this project is to design, develop, and test a complete, end-to-end prototype of the Automated Library Assistant.

The system's boundaries include:

- **User Functionality:** User login, logout, password change, and password reset.
- **Core Library Functions:** Securely borrowing and returning books using a two-factor (digital + physical) authentication system.
- **User Dashboard:** A personal portal to view current loans, borrowing history, and account settings.

- **Machine Learning:** A hybrid (content + collaborative) recommendation system and a predictive model for late return risk.
- **AI Support:** A hybrid chatbot that can answer database-specific queries (e.g., "Is book X available?") and general knowledge questions (via Gemini API).
- **Fine Management:** A complete, recurring fine calculation system integrated with the PayPal sandbox for payment processing.
- **Hardware Integration:** A physical IoT kiosk (Arduino + RFID + LCD) that communicates with the backend via a Python bridge script to provide physical authentication and user feedback.
- **Admin Functionality:** A secure admin dashboard to manage users (add, deactivate) and books (add, search).

Out of Scope:

- This project does not include a full inventory management system (e.g., stock-taking or real-time shelf location).
- The payment system is integrated with the PayPal sandbox only and is not configured for live, real-money transactions.
- The system does not send automated email or SMS notifications (this is left for future work).

4.3 Objectives

1. To develop a secure, low-cost IoT kiosk using Arduino and an RFID reader for physical user authentication for transactions.
2. To design and implement a robust backend server using Python and the Flask framework to manage all application logic, REST APIs, and database interactions.
3. To create a dynamic and intuitive frontend web portal using HTML, Tailwind CSS, and JavaScript that allows users to manage their accounts, transactions, and preferences in real-time.
4. To build a hybrid book recommendation system using collaborative filtering and content-based models to enhance user discovery.
5. To develop a predictive model using a RandomForestClassifier to identify borrowed books that are at a high risk of being returned late.
6. To implement an intelligent chatbot using a rule-based system and the Google Gemini API to provide 24/7 user assistance.
7. To design a comprehensive and secure admin dashboard for complete management of users, books, and library operations.

8. To integrate a complete payment gateway sandbox (PayPal) for managing and settling overdue fines in USD.
9. To provide real-time physical feedback to the user via an I2C LCD display connected to the IoT kiosk.
10. To implement a secure password management system, including "Forgot Password" and "Change Password" functionalities.
11. To impose a borrowing limit (max 2 books) to ensure fair resource distribution.

CHAPTER - 5

PROJECT WORK UNDERTAKEN

This chapter details the specific methodology and logic used to build the "Automated Library Assistant." It covers the system's architecture, database design, and the implementation of its core features.

5.1 Feasibility Study

Feasibility study is made to see if the project on completion will serve the purpose of the organization for the amount of work, effort and the time that spend on it. Feasibility study lets the developer foresee the future of the project and the usefulness. A feasibility study of a system proposal is according to its workability, which is the impact on the organization, ability to meet their user needs and effective use of resources. Thus, when a new application is proposed it normally goes through a feasibility study before it is approved for development.

1. **Technical Feasibility:** The project is technically sound. It relies on a stack of mature, open-source, and well-documented technologies, including **Python, Flask, MySQL, and Arduino**. The required hardware components are low-cost and readily available, ensuring that the technical implementation is straightforward and achievable.
2. **Operational Feasibility:** The system is designed for a tech-savvy user base (students) and integrates smoothly into existing library functions. The web portal is intuitive, and the "scan-to-authorize" workflow is simple, requiring minimal training for both students and staff. This ensures the system will be easily adopted and used effectively.
3. **Economic Feasibility:** This project is highly economical due to its high benefit-to-cost ratio. The development relies entirely on free, open-source software, eliminating licensing costs. The only expenditure is for minimal, low-cost hardware. The benefits, including increased staff efficiency and an improved user experience, far outweigh this minor investment.
4. **Schedule Feasibility:** The project schedule is feasible due to its modular design. The core components (backend, IoT kiosk, frontend, and ML models) can be developed in distinct phases. This approach allows for a flexible and manageable timeline, ensuring the project can be completed successfully within an academic semester.

5.2 System Architecture

The project is built on a distributed, multi-component architecture.

- **User:** The user interacts with the system in two ways: through the Web Frontend (browser) and the IoT Kiosk (RFID card).
- **Web Frontend:** A client-side application (HTML/JS/Tailwind) that runs in the user's browser. It has no direct access to the database and communicates only with the Flask Backend via a REST API.
- **Flask Backend Server:** The central "brain" of the application. It serves the frontend files and provides API endpoints for all logic. It is the only component with permission to read/write to the database.
- **IoT Kiosk:** A physical station, composed of an Arduino (handling hardware) and a Python Kiosk Bridge script (handling communication). This bridge acts as a secure client, calling the Flask API on behalf of the hardware.
- **MySQL Database:** The persistent storage for all application data.

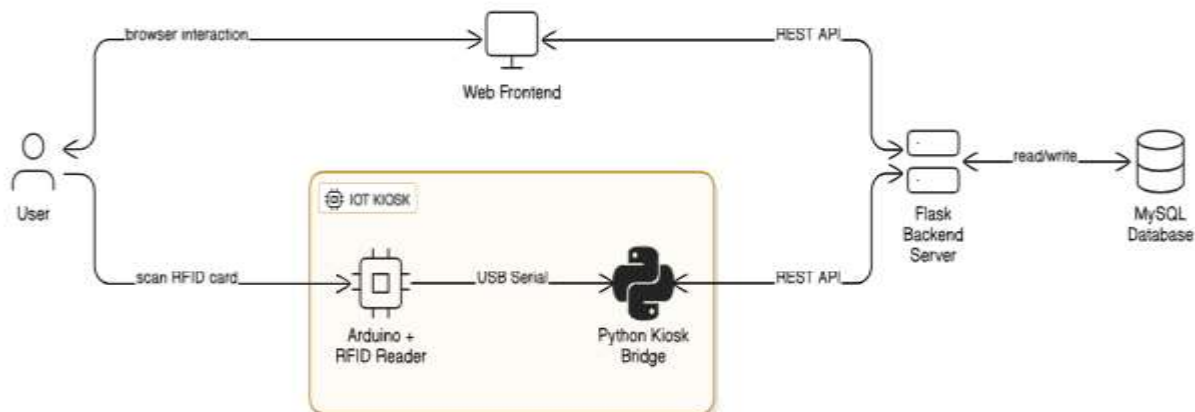


Fig 5.1

5.3 Database Design (ER Diagram)

The database is designed with four core tables: users, books, transactions, and payment_log. This relational structure (ER Diagram) ensures data integrity.

- **users:** Holds all user information, credentials, roles (admin or user), and their is_active status.

Column	Type	Key	Description
user_id	varchar(PK)	PK	The user's unique ID (e.g., "21BCS001").
name	varchar		The user's full name.
password	varchar		The user's hashed password.
email	varchar		The user's email, used for password resets.
rfid_uid	varchar		The unique ID from their physical RFID card.
role	varchar		"user" or "admin".
is_active	boolean		true if active, false if deactivated.

Table 5.1

- **books:** Catalogs all library materials

Column	Type	Key	Description
book_id	varchar(PK)	PK	The book's unique ID (e.g., ISBN).
title	varchar		The title of the book.
author	varchar		The author of the book.
status	varchar		"Available" or "Issued".
Image-URL-M	varchar		A URL to the book's cover image.

Table 5.2

- **transactions:** The central table, linking users and books. A new row is created for every borrow event.

Column	Type	Key	Description
transaction_id	int(PK)	PK	A unique ID for this specific loan.
user_id	varchar(FK)	FK	Foreign key referencing users.user_id.
book_id	varchar(FK)	FK	Foreign key referencing books.book_id.
issue_date	datetime		The exact date and time the book was borrowed.
due_date	datetime		The date and time the book is due.
return_date	datetime		NULL if out, or the date it was returned.
fine_amount	decimal		The last calculated fine amount.
fine_status	varchar		NULL, "Pending", or "Paid".
fine_paid_until	date		The date until which the fine has been paid.

Table 5.3

- **payment_log:** This table acts as an audit trail for payment attempts, linking a user_id to a payment_request_id from PayPal to ensure the capture request is secure and valid.

Column	Type	Key	Description
log_id	int(PK)	PK	A unique ID for the log entry.
user_id	varchar(FK)	FK	Foreign key referencing users.user_id.
amount	decimal		The amount (USD) of the payment attempt.
payment_request_id	varchar		The Order ID generated by PayPal.
payment_id	varchar		The final Payment ID after capture.
status	varchar		"Pending", "Paid", or "Failed".
created_at	datetime		The timestamp when the payment was initiated.

Table 5.4

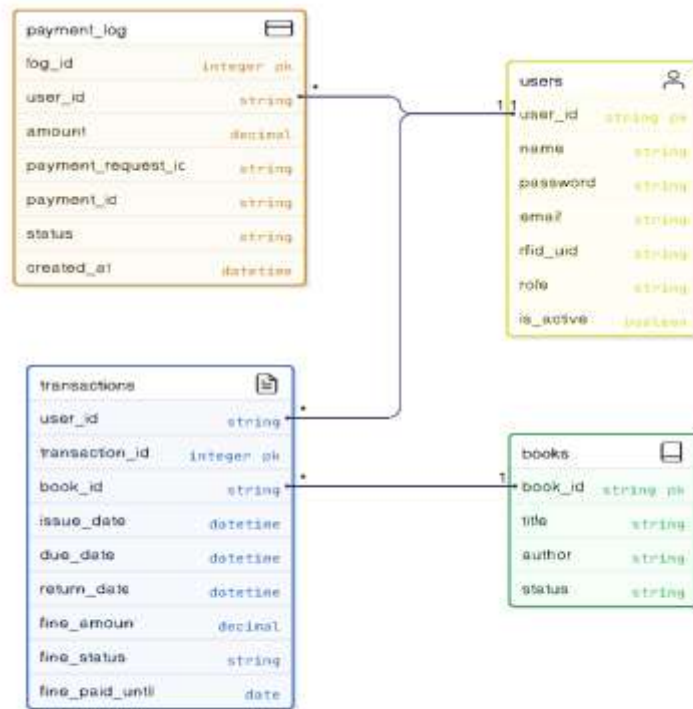


Fig 5.2

5.4 Secure IoT Transaction Workflow

The diagram below illustrates the end-to-end process for a user performing a transaction, showing the interaction between the web portal, the physical kiosk, and the backend server.

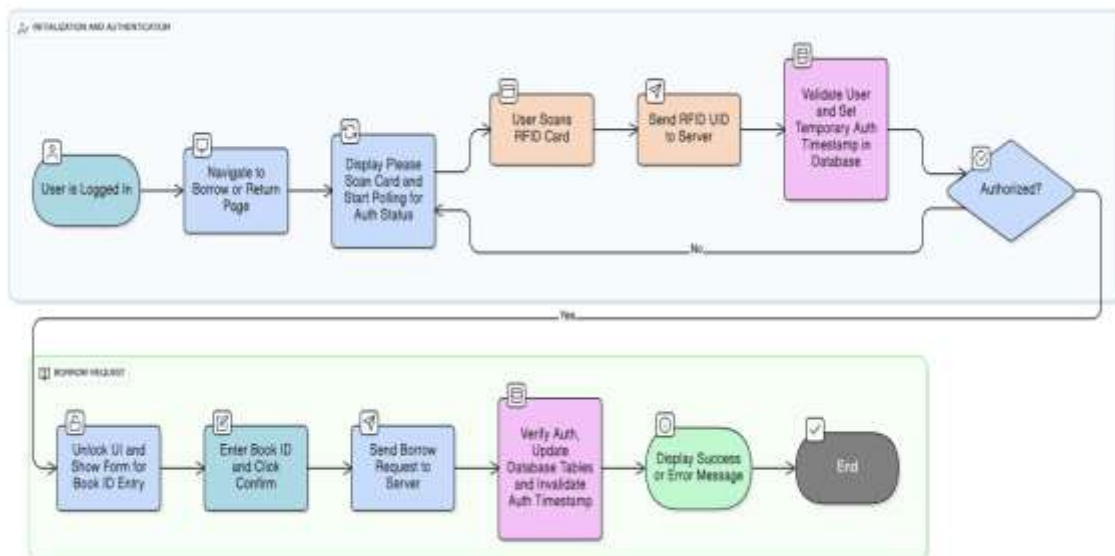


Fig 5.3

Description of the Flow:

1. **User Login & Navigation:** The process begins with the user logging into the web portal with their User ID and password. They navigate to the "Borrow a Book" or "Return a Book" page.
2. **UI Locks & Polling:** The web page immediately locks the UI and displays a "Please scan your card" prompt. In the background, script.js starts polling the /api/user/status endpoint every 2 seconds, repeatedly asking the server, "Is this user authorized yet?"
3. **Physical Authentication (RFID):** The user scans their physical RFID card at the IoT kiosk. The Arduino reads the card's unique ID.
4. **Hardware-to-Software Bridge:** The Arduino sends the UID over the USB serial port to the kiosk_bridge.py script.
5. **Kiosk-to-Server API Call:** The kiosk_bridge.py script sends this UID to the Flask server's /api/authorize endpoint.
6. **Server-Side Validation:** The authorize() function in app.py:
 - Finds the user with that rfid_uid.
 - If not found, it returns a {"status": "error", "message": "RFID Not Found"}.
 - If the user is is_active = false, it returns {"status": "error", "message": "Account Inactive"}.
 - If found and active, it updates that user's record, setting auth_timestamp = NOW() (the current time).
 - It returns {"status": "success", "user_name": "John"}.
7. **Physical Feedback (LCD):** The kiosk_bridge.py script receives the JSON response.
 - On "success", it sends the command SUCCESS:John back to the Arduino, which displays "Welcome, John :)" on the I2C LCD.
 - On "error", it sends ERROR:RFID Not Found, which the Arduino displays.
8. **UI Unlocking:** On its next 2-second check, the web app's poll to /api/user/status now returns {"authorized": true}. The JavaScript immediately stops polling, hides the "Please scan..." prompt, and shows the "Step 2: Identify Book" form.
9. **Book Identification (Barcode/Manual):** The user enters the book_id (e.g., an ISBN) either by typing or by using the **on-page QR/Barcode scanner** (powered by the html5-qrcode library).
10. **Final Transaction API Call:** The user clicks "Confirm." script.js sends the bookId to the Flask server (e.g., POST /api/book/borrow).

11. Server-Side Transaction Logic: The `perform_transaction()` function is called. This is the final, critical step:

- **Verifies Auth:** Checks if the `auth_timestamp` is *still valid* (i.e., less than 2 minutes old).
- **Invalidates Auth:** Immediately sets the `auth_timestamp` to NULL to prevent reuse.
- **Checks Business Rules:** It checks all rules:
 - Does the user have 2 or more books already?
 - Does the user have any pending fines?
 - Does the user have any overdue books?
- **Processes:** If all checks pass, it updates the books table (`status = 'Issued'`) and creates a new row in the transactions table.

12. Final Confirmation & E-Receipt: The server sends a `{"status": "success", "message": "Book issued successfully!", "receipt_url": "/api/receipt/pdf/339"}` JSON response. The JavaScript displays the success message and shows a "Download E-Receipt" button, which links to the dynamic PDF URL.

5.5 Fine Calculation & Payment Methodology

This project implements a sophisticated recurring fine system. A user is not absolved of a fine simply by paying once if they continue to hold the book.

1. **Daily Check:** When the user dashboard is loaded, the system scans all active loans (`return_date` IS NULL).
2. **Calculate Fine Start Date:** For each overdue book (`today > due_date`), the system determines the correct start date for the fine:
 - If `fine_paid_until` is NULL (the fine has never been paid), the `fine_start_date` is the `due_date`.
 - If `fine_paid_until` has a date (the fine *has* been paid), the `fine_start_date` is the `fine_paid_until` date.
3. **Calculate New Fine:** The system calculates `days_late = today - fine_start_date`. The new fine is `days_late * FINE_PER_DAY_USD` (\$0.10 per day). This is the amount shown to the user.
4. **Create Order:** When the user clicks the PayPal button, the `/api/paypal/create_order` function "commits" these new fines to the database (setting `fine_status = 'Pending'` and `fine_amount = [new_fine]`) and generates a PayPal order for the total.

5. **Capture Order:** When payment is approved, the `/api/paypal/capture_order` function:
- Validates the `orderId` against the `payment_log` table for security.
 - Makes a secure call to PayPal to capture the funds.
 - If successful, it updates the database: it sets the `fine_status` to `NULL` (clearing the "pending" state) and, most importantly, **sets `fine_paid_until` to the current date**. This ensures that if the user logs in tomorrow, a new fine of \$0.10 will be calculated for the new day.

5.6 Security & Password Management Methodology

To enhance security and user convenience, two separate password management flows were implemented.

5.6.1 Change Password Flow

This feature allows a logged-in user to change their own password from the "Settings" tab.

1. **Input:** User provides "Old Password" and "New Password."
2. **API Call:** Frontend sends this data to the `/api/user/change_password` endpoint.
3. **Verification:** The server checks if the `user_id` (from the session) and the `old_password` match a user in the database.
4. **Action:** If they do not match, an "Incorrect old password" error is returned. If they do match, the server updates the user's record with the new password.

5.6.2 Forgot Password Flow

This feature allows a logged-out user to reset their password from the login screen.

1. **Input:** User provides their "User ID," their registered "Email," and a "New Password."
2. **API Call:** Frontend sends this data to the `/api/forgot_password` endpoint.
3. **Verification:** The server performs a critical security check: `SELECT user_id FROM users WHERE user_id = %s AND email = %s`. This ensures a user cannot change another person's password without also knowing their private email address.
4. **Action:** If no user is found, an error is returned. If they do match, the server updates the user's password, allowing them to log in.

5.7 Machine Learning Model Development

The project includes two distinct machine learning systems that were pre-trained in a Jupyter Notebook and saved to .joblib files.

5.7.1 Data Collection & Preprocessing

Data was sourced from the "Book-Crossings" dataset, which contains three files:

- **BX-Users:** Contains user IDs and locations.
- **BX-Books:** Contains book ISBN, title, author, and image URLs.
- **BX-Book-Ratings:** Contains user IDs, ISBNs, and ratings from 0-10. This data was heavily preprocessed. Explicit ratings (1-10) and implicit ratings (0) were combined. The data was filtered to include only popular books and active users to reduce sparsity.

5.7.2 Model 1: Hybrid Recommendation System

A simple "New User" or "Cold Start" problem exists where a user has no ratings, making standard recommendations impossible. Our system solves this with a hybrid approach.

- **Content-Based Filtering:**
 1. Book titles and authors were combined into a single text string.
 2. The sentence-transformers/all-MiniLM-L6-v2 model was used to convert this text into 384-dimension vectors.
 3. These vectors were saved in an embedding matrix (content_embeddings.joblib).
 4. When a user likes a book, this system finds the 5 most similar books by calculating the cosine_similarity between their vectors. This is excellent for recommending niche or new books.

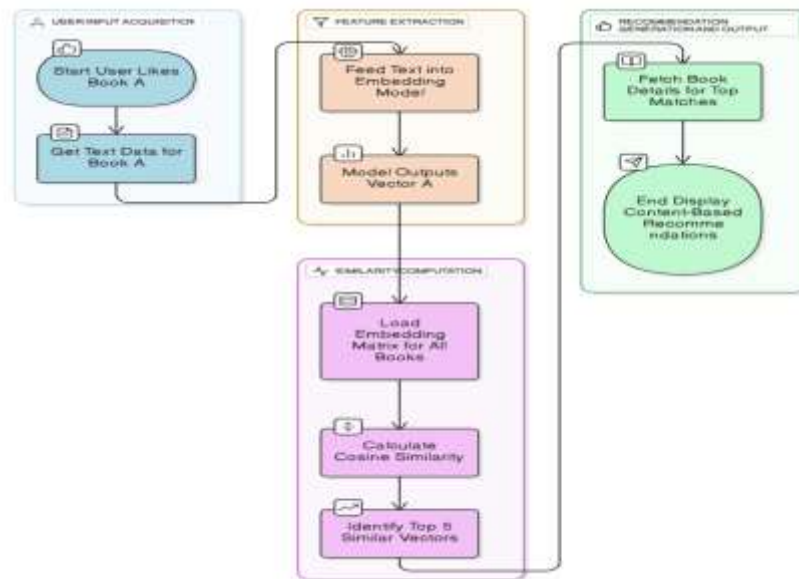


Fig 5.4

- **Collaborative Filtering:**

1. A user-item pivot table was created from the ratings data, with users as rows and book titles as columns.
2. A NearestNeighbors model from Scikit-learn was trained on this matrix.
3. This model finds users with similar rating histories. It recommends books that "similar users" liked but the current user has not yet read.

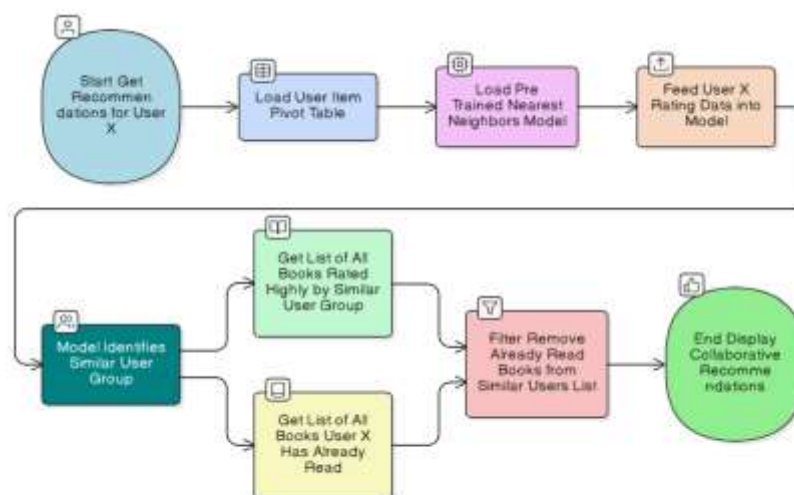


Fig 5.5

- **Hybrid Logic:** The `get_book_recommendations` function first gets 10 content-based and 10 collaborative-based recommendations. It then blends them, removes duplicates, and filters out any books the user has already read, presenting the top 5 results.

5.7.3 Model 2: Late Return Prediction

This model predicts the *probability* of a book being returned late.

1. **Feature Engineering:** The historical transactions table was used. The most important feature was `day_of_week_borrowed` (extracted from `issue_date`). A `loan_duration_days` (14) was also used.
2. **Model Training:** A `RandomForestClassifier` was trained on this data. A Random Forest is an "ensemble" model that combines many decision trees to make a more accurate and robust prediction.
3. **Implementation:** The `predict_late_return` function takes a user's new loan and passes its features to the trained model. It returns the probability of lateness (e.g., 75%), which is then displayed in the UI as a "High Risk" tile.

5.8 AI Chatbot Methodology

The chatbot provides 24/7 user assistance. It uses an efficient hybrid strategy to provide fast, accurate answers.

- **Chatbot Rule-Based Intents:** The system first checks for specific keywords to provide fast, accurate, database-driven answers.

Intent	Keywords	Action
Book Availability	("is", "do you have") + "Book Title"	Queries books table for status.
Library Hours	"hours"	Returns a hard-coded string.
Fine Policy	"fine"	Returns a hard-coded string.
General Knowledge	(All other messages)	Sends the prompt to the Google Gemini API.
Intent	Keywords	Action

Table 5.5

- **Chatbot Logic Flow**

1. **Rule-Based Intent Parsing:** When a message is received, the `get_chatbot_response` function first checks for the keywords in Table 4.5.
2. **Database-Specific Intents:** If the message contains ("is", "do you have") AND a title in quotes (e.g., "Metamorphosis"), the system queries the books table directly and gives a factual answer like, "Sorry, 'Metamorphosis' is currently Issued."
3. **AI Fallback (Gemini):** If the message does not match any predefined rules, it is passed to the Google Gemini API. The API is given a prompt to act as a "friendly and helpful library assistant" and answers the general knowledge question (e.g., "What is Python?"). This ensures the user always gets a useful response.

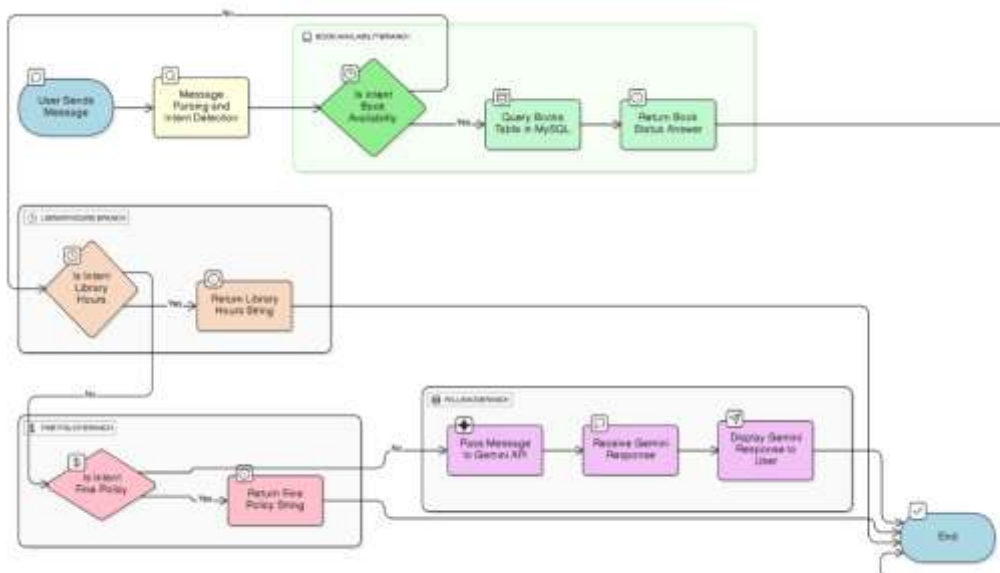


Fig 5.6

CHAPTER – 6

RESULTS AND DISCUSSION

This chapter details the implementation of the project's components and showcases the final, working user interfaces.

6.1 Hardware Implementation

The IoT Kiosk consists of three main components connected to an Arduino Uno:

- **MFRC522 RFID Reader:** Connected via the SPI protocol.
- **I2C LCD Display:** A 16x2 display connected via the I2C protocol.
- **Arduino Uno:** This acts as the central microcontroller, programmed in C++ to read from the RFID reader and write to the LCD.

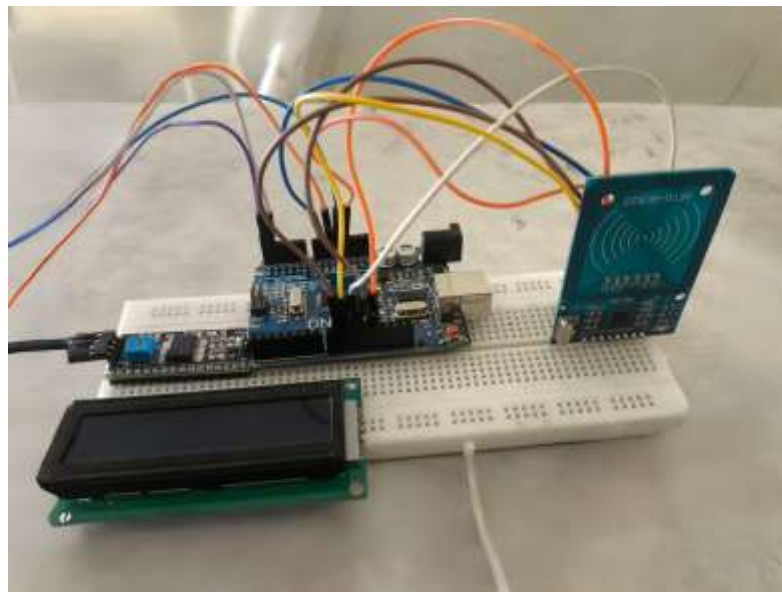
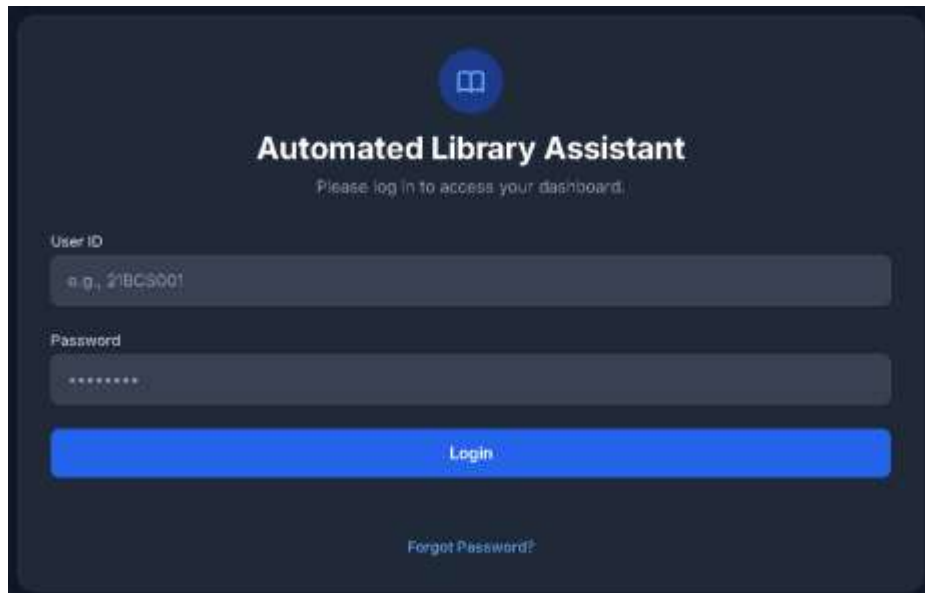


Fig 6.1

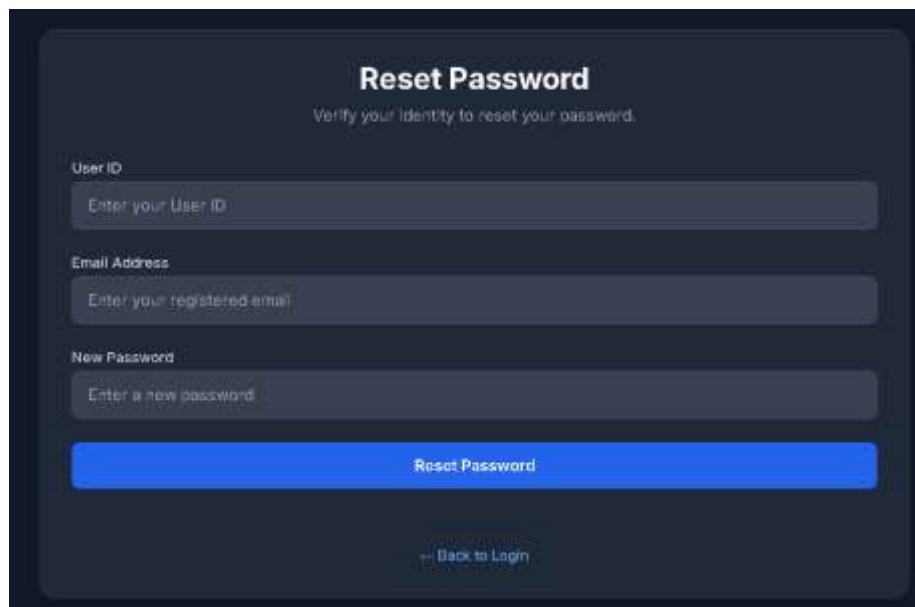
6.2 Software Implementation: Login, Security & Theme

The user is first presented with a login page. The UI automatically adapts to the system's light or dark mode. A "Forgot Password" link is available for password recovery.



The image shows a login page for the 'Automated Library Assistant'. At the top, there is a blue circular icon with a white book symbol. Below it, the title 'Automated Library Assistant' is displayed in white, followed by the instruction 'Please log in to access your dashboard.' in a smaller font. The login form consists of two input fields: 'User ID' with a placeholder 'e.g., 21BCS001' and 'Password' with a placeholder '*****'. Below these fields is a prominent blue 'Login' button. At the bottom of the form, there is a link labeled 'Forgot Password?' in a smaller, lighter font.

Fig 6.2



The image shows a 'Reset Password' page. The title 'Reset Password' is centered at the top in white, with the instruction 'Verify your identity to reset your password.' below it. The form contains three input fields: 'User ID' with a placeholder 'Enter your User ID', 'Email Address' with a placeholder 'Enter your registered email', and 'New Password' with a placeholder 'Enter a new password'. Below these fields is a blue 'Reset Password' button. At the bottom of the form, there is a link labeled 'Back to Login' in a smaller, lighter font.

Fig 6.3

6.3 IoT Kiosk Interface and User Feedback

When the user scans their card, the Python bridge communicates with the server, and the I2C display provides immediate, physical feedback.



Fig 6.4



Fig 6.5

6.4 User Dashboard

After a successful login, the user sees their dashboard. The UI is organized into tabs:

- **My Books:** Shows transaction buttons, payment alerts, and a card-based view of current loans.
- **Recommendations:** Shows AI-generated book recommendations.
- **Settings:** Allows the user to change their password.

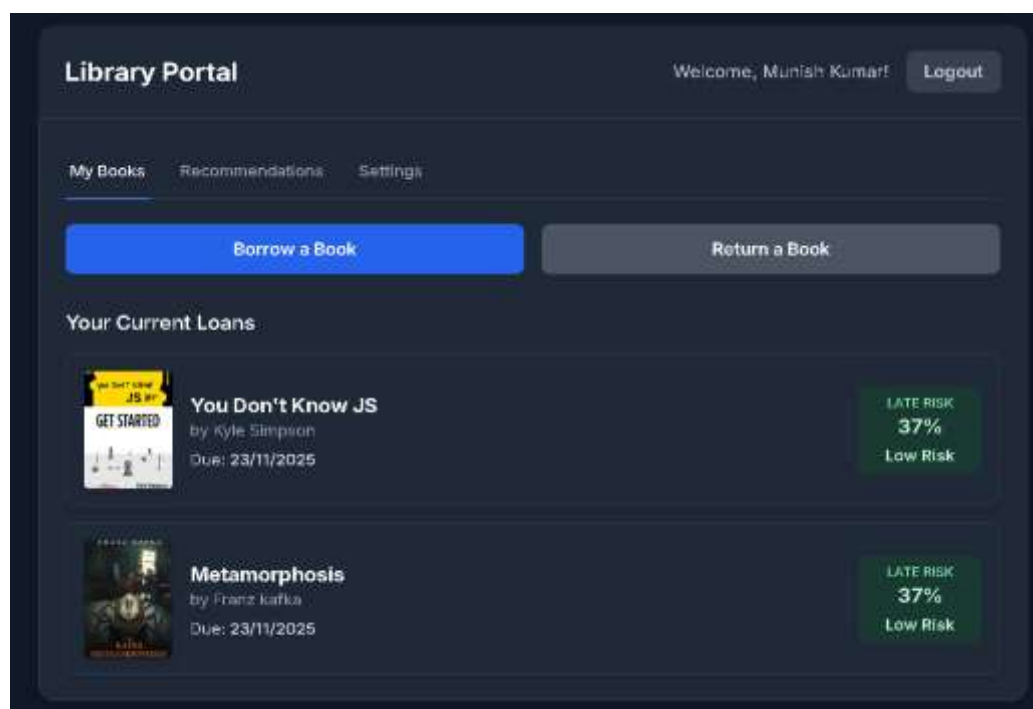


Fig 6.6

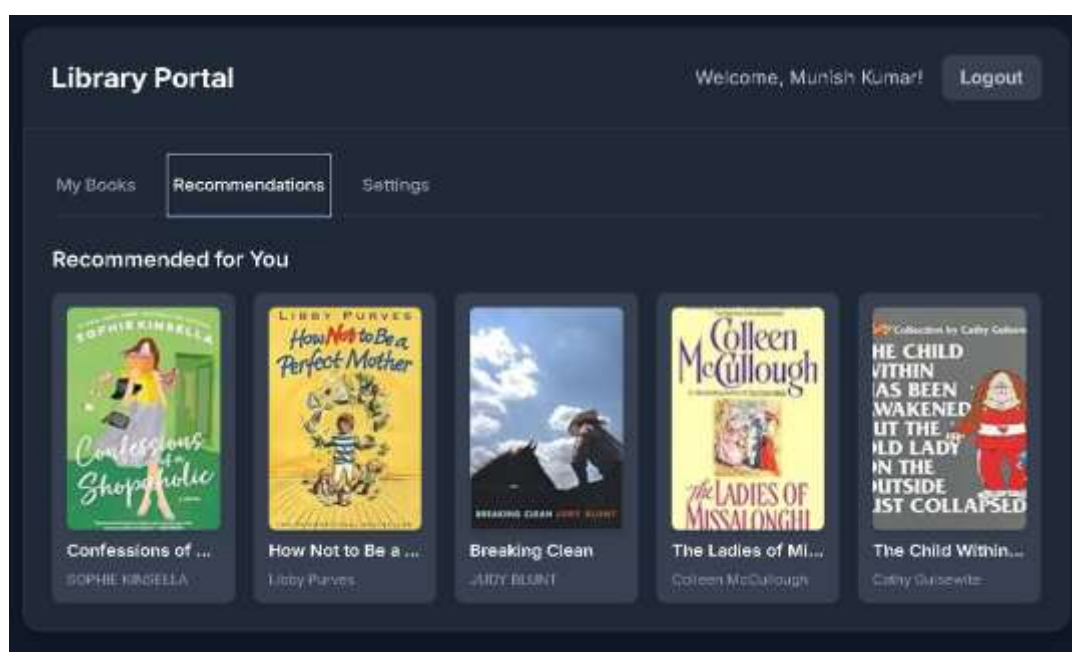


Fig 6.7

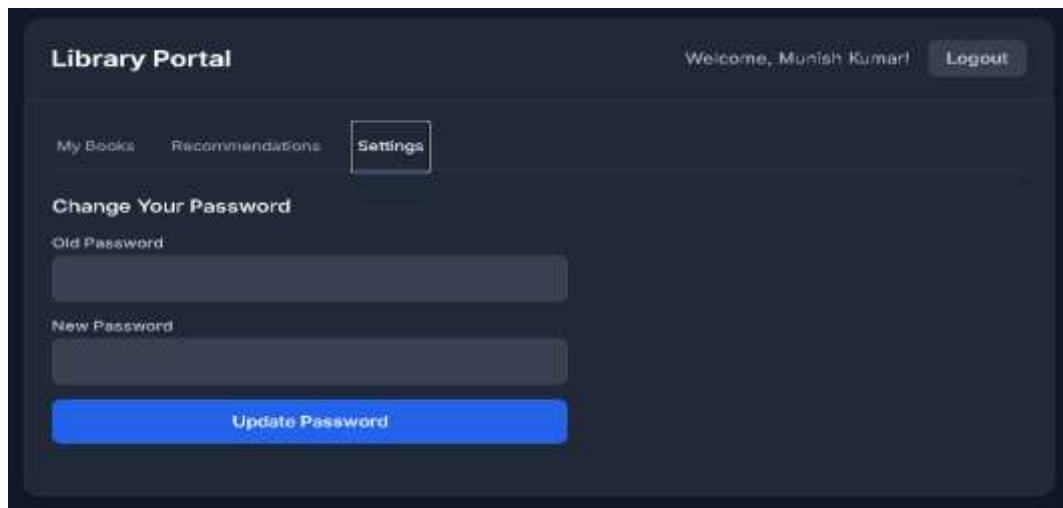


Fig 6.8

6.5 Loan Status & Risk Prediction Tiles

The "My Books" tab uses a "tile" system to show loan status:

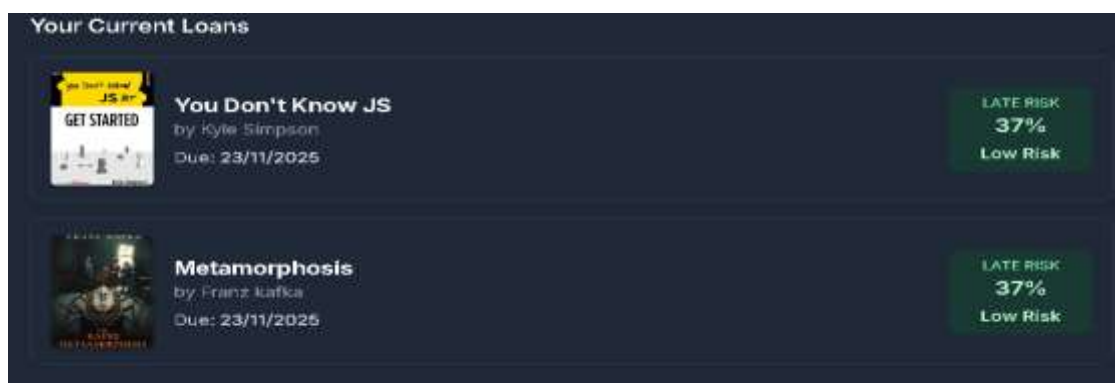


Fig 6.9

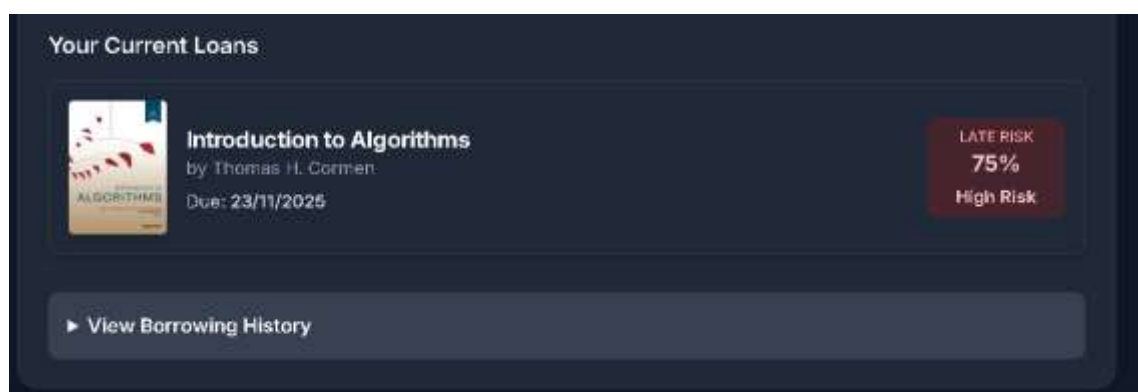


Fig 6.10

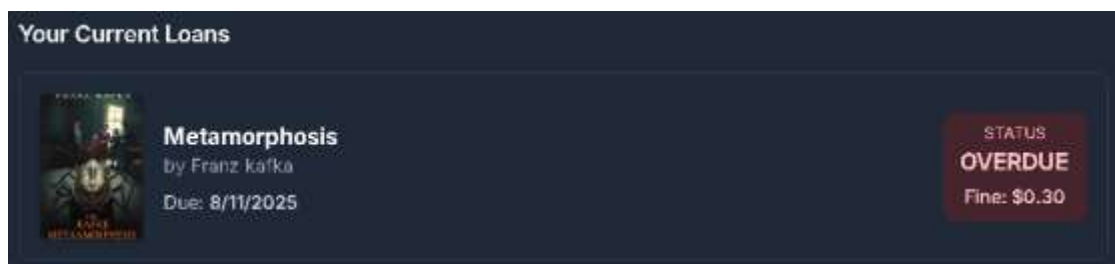


Fig 6.11

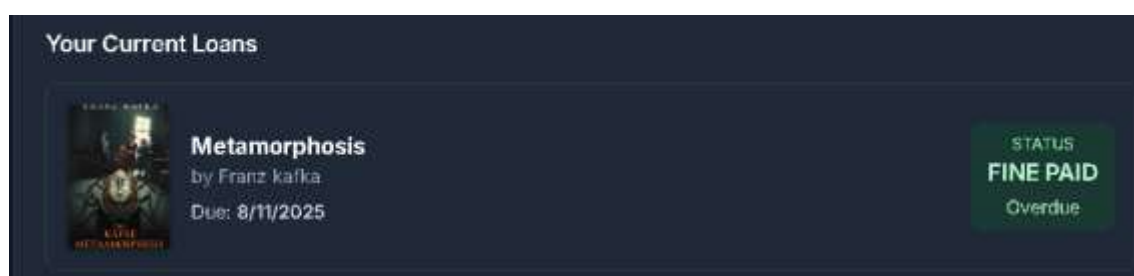


Fig 6.12

6.6 Fine Payment System (PayPal)

When a user has an outstanding fine, a high-visibility alert box appears above their loans. This box contains the PayPal payment buttons. After payment, the dashboard reloads, the alert box disappears, and the book cards update to "Fine Paid."

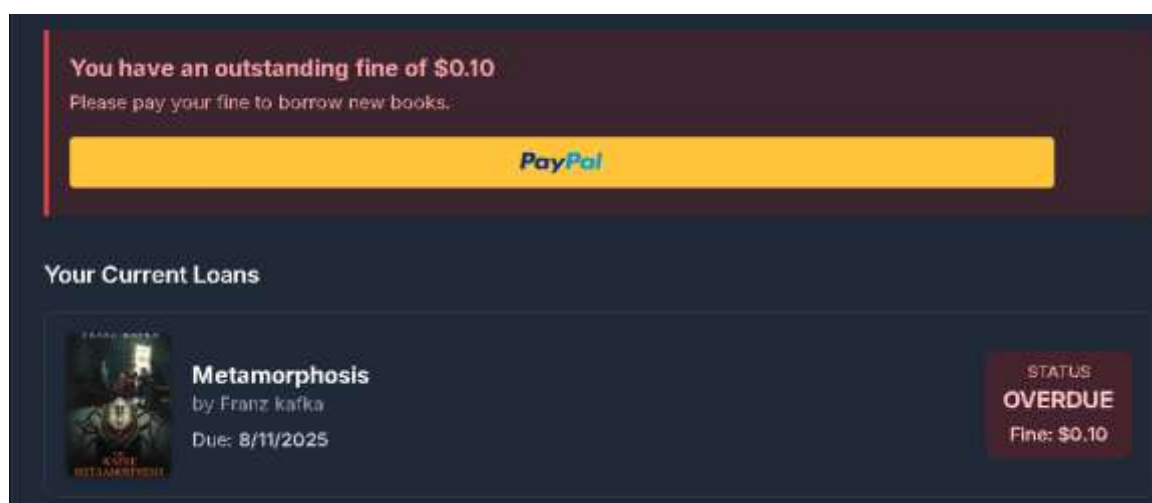


Fig 6.13

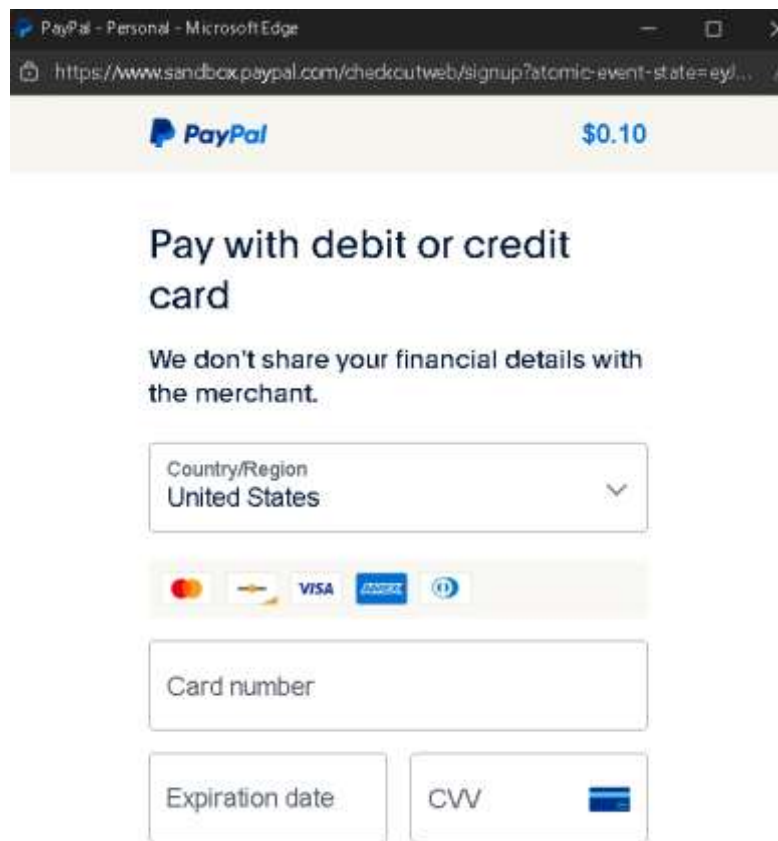


Fig 6.14

6.7 AI Chatbot

The chatbot icon is available on the user dashboard. It can answer both library-specific and general knowledge questions.

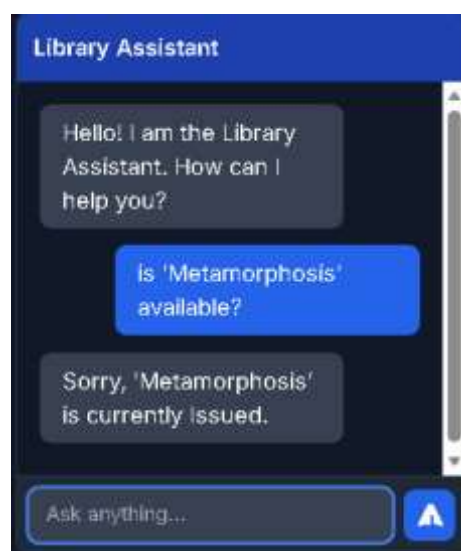


Fig 6.15

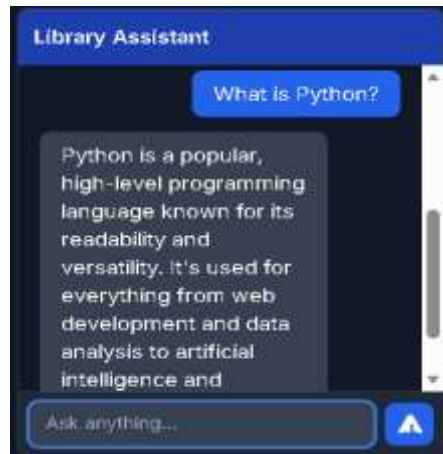


Fig 6.16

6.8 Admin Dashboard

The admin dashboard provides a full control panel for library management.

- **Overview:** Shows all overdue books and those due soon.
- **User Management:** Forms to add new users and deactivate existing ones.
- **Book Management:** Forms to add new books and search the catalog.

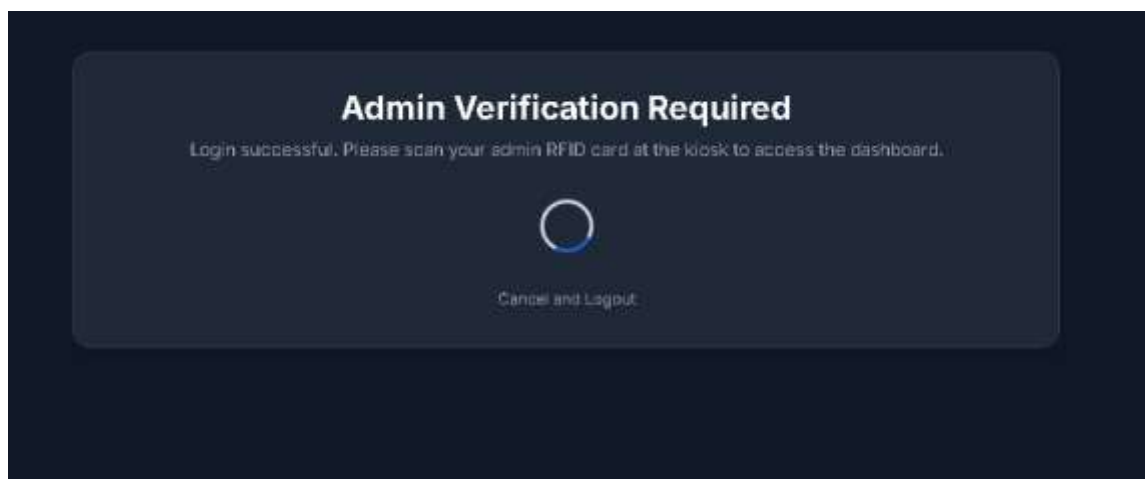


Fig 6.17

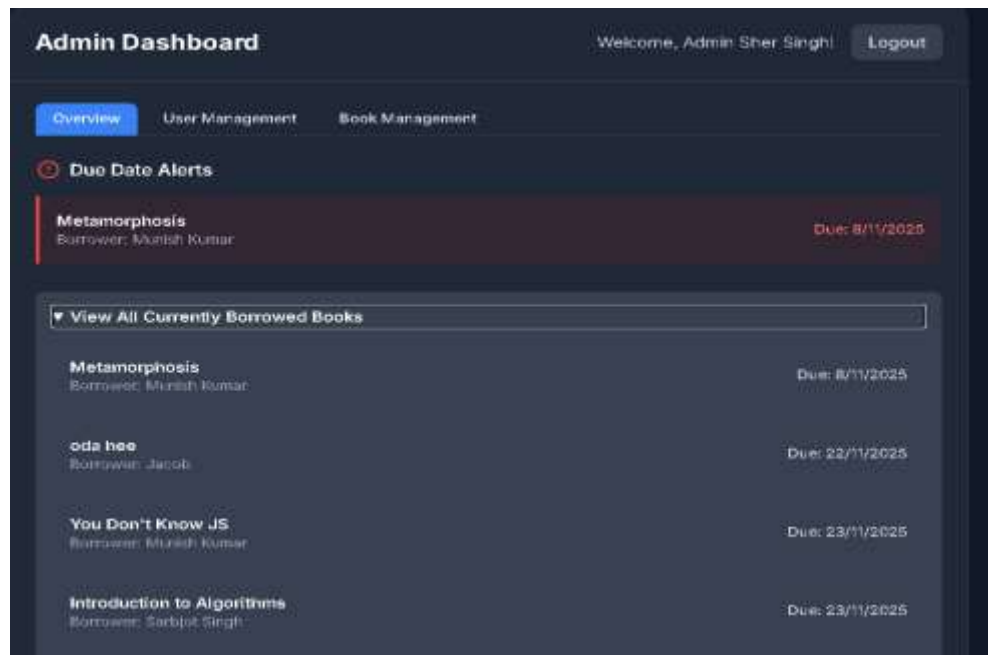


Fig 6.18

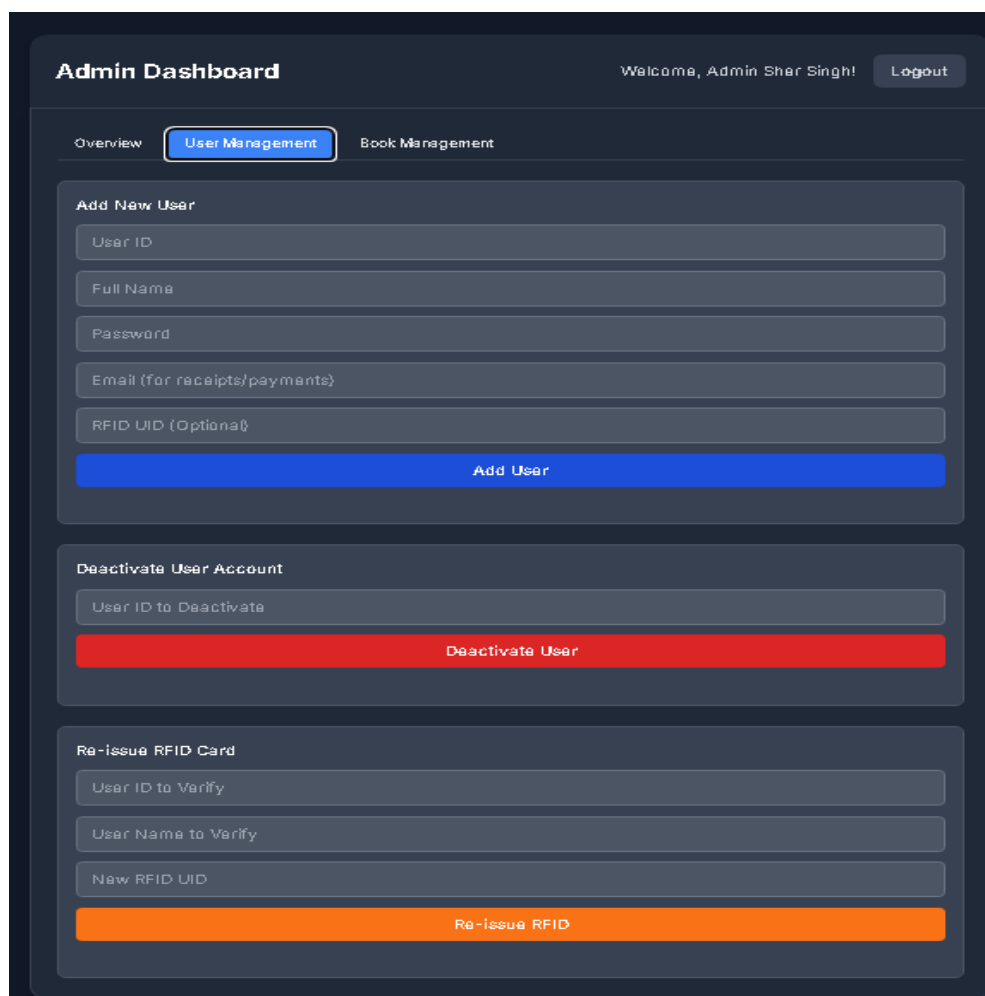


Fig 6.19

The image shows a dark-themed Admin Dashboard. At the top left is the title 'Admin Dashboard'. At the top right is a welcome message 'Welcome, Admin Sher Singh!' and a 'Logout' button. Below the header is a navigation bar with three tabs: 'Overview', 'User Management', and 'Book Management' (which is highlighted in blue). The main content area is divided into two sections. The first section is titled 'Add New Book' and contains three input fields: 'Book ID (ISBN)', 'Book Title', and 'Author'. Below these fields is a large blue button labeled 'Add Book'. The second section is titled 'Search Book Status' and contains a single input field labeled 'Search by title...'.

Fig 6.20

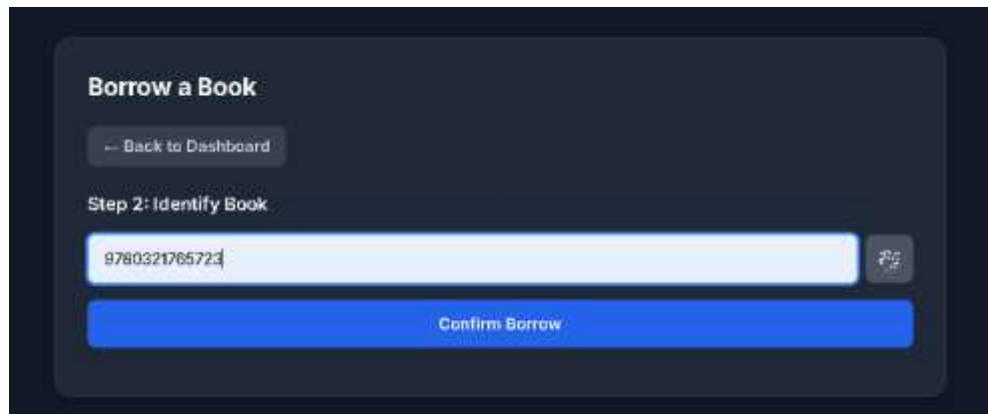
6.9 Transaction Authorization Flow & E-Receipt

- When a user clicks "Borrow" or "Return," they must first authorize at the kiosk.

The image shows a screen titled 'Borrow a Book'. In the top left corner, there is a button with a left-pointing arrow and the text 'Back to Dashboard'. In the center of the screen, the text 'Step 1: Authorize Transaction' is displayed above the instruction 'Please scan your RFID/ID card at the library kiosk to proceed.'. Below this text is a circular loading indicator consisting of a blue ring and a small blue dot.

Fig 6.21

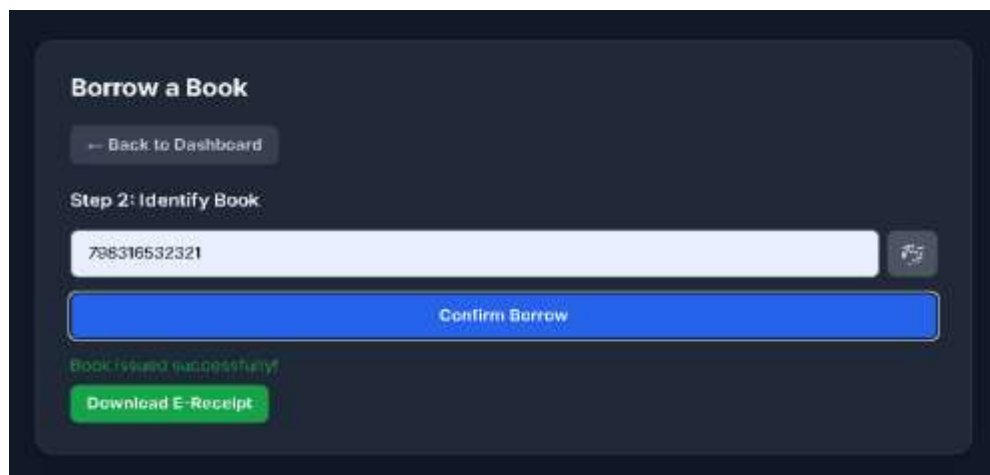
- The system then unlocks, allowing the user to enter the Book ID. The HTML5-qrcode library provides the option to use the device's camera as a barcode/QR code scanner.



The screenshot shows a dark-themed web interface titled "Borrow a Book". At the top left, there is a link that says "← Back to Dashboard". Below this, the heading "Step 2: Identify Book" is displayed. A light blue input field contains the text "9780321765723". To the right of the input field is a small square icon with a camera symbol. Below the input field is a large blue button with the text "Confirm Borrow".

Fig 6.22

- Upon a successful borrow, the backend generates a dynamic PDF e-receipt, which the user can download.



The screenshot shows the same "Borrow a Book" interface. The input field now contains the text "798316532321". Below the "Confirm Borrow" button, a green message "Book Issued successfully!" is displayed. Below this message is a green button with the text "Download E-Receipt".

Fig 6.23

CHAPTER - 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The "Automated Library Assistant" successfully integrates IoT, web technologies, and machine learning to create a modern and efficient library management system. The project effectively addresses the key limitations of traditional systems by automating manual processes, providing a secure and seamless user experience, and unlocking the power of data analytics.

The implementation of the secure RFID transaction system, the hybrid recommendation engine, the late return predictor, and the intelligent Gemini-powered chatbot demonstrates a comprehensive solution that significantly enhances value for both library users and administrators. The addition of a recurring fine payment system via PayPal and a physical I2C display for user feedback makes the system a feature-complete prototype.

The modular architecture ensures the system is not only functional and robust but also scalable for future enhancements, paving the way for a smarter, more responsive library ecosystem.

7.2 Future Scope

- While this project provides a robust foundation, several features could be added to further enhance its capabilities:
- **Data Logging & Analytics:** Log all chatbot queries and fine payment events to a new database table. This data could be used to build a new admin-facing analytics dashboard to visualize peak borrowing times, most-fined books, and common user questions.
- **Email Notifications:** Integrate an email service (like SendGrid) to automatically email users a copy of their e-receipt upon borrowing or a warning message when their book is due soon.
- **Mobile Application:** Adapt the web frontend into a progressive web app (PWA) or a native mobile app for better accessibility, allowing users to check their due dates from their phones.
- **Advanced Inventory Management:** Utilize the RFID reader in a "wand" mode, allowing librarians to walk the shelves and perform a full inventory check against the database in real-time.
- **Self-Service Kiosk:** Expand the kiosk to include a barcode scanner, allowing users to complete the entire borrow/return process at the kiosk without needing to use the web portal.

7.3 Recommendations of the Study

Based on the successful design, implementation, and testing of the "Automated Library Assistant" prototype, the following recommendations are proposed to the college administration and library staff:

- **Formal Adoption and Phased Rollout:** It is recommended that the college library formally adopt this system to replace the current legacy management practices. A phased rollout, starting with first-year students to register their RFID cards, would ensure a smooth transition and allow for feedback collection.
- **Policy Update for Digital Fine Management:** The study successfully implemented a recurring, automated fine system with a PayPal gateway. It is recommended that the library officially update its fine policy to accept (and eventually prefer) digital payments, which would reduce administrative overhead and be more convenient for students.
- **Implement Data-Driven Collection Development:** The project proves that borrowing patterns and user data can be effectively captured. It is strongly recommended that library staff be trained to use the admin dashboard's data to make budget and purchasing decisions based on empirical evidence (e.g., popular books, high-demand authors) rather than intuition alone.
- **Proactive Student Support Using Predictive Analytics:** The late return prediction model was shown to be effective. It is recommended that library staff use this tool to *proactively* engage with students whose loans are flagged as "High Risk," offering extensions or reminders to improve return rates and student engagement.
- **Staff Re-allocation and Training:** By automating manual checkout and support queries (via the chatbot), this system frees up valuable staff time. It is recommended that staff be retrained to focus on higher-value tasks, such as providing research assistance, curating digital collections, and managing the new "Student Success Platform".
- **Integration of Kiosk as a Standard Security Point:** The IoT kiosk provides a secure "two-factor" authentication. It is recommended that this physical authentication method be adopted as a standard security protocol for all high-privilege actions, including admin access and password resets, to enhance overall campus digital security.

BIBLIOGRAPHY

- [1] Kaur, A., & Kaur, A. (2018). RFID-Based Smart Library Management System. *International Journal of Computer Applications*, 180(14), 21-25.
- [2] Sarmah, D., & Bhuyan, M. H. (2015). A Web-based book recommendation system using collaborative filtering. *Proceedings of the 2015 International Conference on Green Computing and Internet of Things (ICGCIoT)*.
- [3] Gudadhe, M., Singhal, K., & Wankhade, C. (2010). A new approach for library management system using RFID technology. *2010 International Conference on Computer and Communication Technology*.
- [4] Patel, R., & Shah, N. (2019). An IoT-Based Framework for a Modern Smart Library. *International Journal of Innovative Technology and Exploring Engineering*, 8(9), 123-128.
- [5] O'Connor, S., & Brien, L. (2017). Implementing a Hybrid Book Recommendation System in an Academic Library Portal. *Journal of Web Librarianship*, 11(2), 85-102.
- [6] Chen, Y., & Li, X. (2020). Predicting Overdue Books in Academic Libraries Using Machine Learning Classification. *Information Discovery & Delivery*, 48(3), 155-164.
- [7] Ali, M. Y., & Kumar, S. (2021). Design and Implementation of a Python-Flask Based Web Application for University Library Services. *Journal of Engineering and Technology*, 12(1), 45-56.
- [8] Nurcan, A., & Halil, I. (2019). A Chatbot for Library Services: A Case Study on the Sakarya University Library. *Journal of Librarianship and Information Science*, 51(4), 1043-1055.
- [9] Sonawane, P., & Pardeshi, S. (2016). RFID and Web-based an Integrated Library Management System. *International Journal of Computer Science and Information Technologies*, 7(2), 654-657.
- [10] Zhao, Q., & Sung, A. H. (2014). Data mining and machine learning in the context of library and information science. *Journal of the Association for Information Science and Technology*, 65(10), 1943-1958.
- [11] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 3982–3992.
- [12] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(Oct), 2825-2830.
- [13] Breiman, L. (2001). Random forests. *Machine learning*, 45(1), 5-32.
- [14] Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331-370

- [15] López-López, A., González, R. L., & Araujo, J. A. (2023). Generative AI in Academic Libraries: A New Era for Information Services. *The Journal of Academic Librarianship*, 49(5), 102750.
- [16] Wolf, T., et al. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38-45.
- [17] McKinney, W. (2010). Data structures for statistical computing in python. *Proceedings of the 9th Python in Science Conference*, 445(1), 51-56.
- [18] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
- [19] Bhowmick, A., & Hazarika, S. M. (2017). An IoT-based smart library management system. *2017 8th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, 1-6.
- [20] H. M., K. S., & G. N., S. (2021). An IoT Based Smart Library System Using Arduino and RFID. *International Journal of Engineering Research & Technology*, 10(7), 486-489.
- [21] Luo, H., & Yang, T. (2022). A hybrid book recommendation system based on knowledge graph and collaborative filtering. *Data Technologies and Applications*, 56(4), 582-598.
- [22] Al-Shboul, B., & Al-Sayyed, R. (2021). The use of artificial intelligence in academic libraries: A review of the literature. *The International Journal of Information, Diversity, & Inclusion*, 5(2), 164-180.
- [23] Shinde, P. P., & Shinde, S. (2020). Predictive Models for Library Management: A Review. *International Journal of Computer Applications*, 174(30), 1-6.
- [24] Wang, H., & Zhang, F. (2021). A Hybrid Recommendation System with Deep Learning for E-commerce. *2021 IEEE International Conference on Consumer Electronics (ICCE)*, 1-6.

Appendix

A. Glossary of Terms

- **API (Application Programming Interface):** A set of rules allowing different software applications to communicate with each other. Used by your web frontend, Flask backend, and PayPal.
- **Collaborative Filtering:** A recommendation technique that predicts what a user will like based on the preferences of *similar users*.
- **Content-Based Filtering:** A recommendation technique that suggests items (books) based on their *own properties* (e.g., title, author) by finding items similar to what the user already likes.
- **Cold Start Problem:** A common issue in recommendation systems where it is difficult to make recommendations for new users or new items (books) that have no historical data.
- **Flask:** A lightweight "micro" web framework for Python, used to build your backend server and API.
- **Hybrid Recommendation System:** A system that combines two or more recommendation techniques (in your case, Collaborative and Content-Based) to improve accuracy and handle issues like the cold start problem.
- **I2C (Inter-Integrated Circuit):** A communication protocol used in your hardware to send data to the LCD display using only two wires.
- **PySerial:** A Python library that allows your `kiosk_bridge.py` script to read data from and write data to the Arduino's USB serial port.
- **RFID (Radio-Frequency Identification):** A technology that uses radio waves to read information stored on a tag (your ID card). Used in your project for secure, physical user authentication.
- **Sentence-Transformers:** A Python library used to create high-quality numerical representations (embeddings) of text, which you use to find semantically similar books.
- **SPA (Single Page Application):** A web application design where all content is loaded on a single HTML page, and JavaScript is used to show/hide different sections, providing a faster user experience.

B. Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
I2C	Inter-Integrated Circuit
IoT	Internet of Things
JS	JavaScript
LCD	Liquid Crystal Display
ML	Machine Learning
PDF	Portable Document Format
RDBMS	Relational Database Management System
RFID	Radio-Frequency Identification
SPA	Single Page Application
UID	Unique Identifier
USD	United States Dollar

Table A.1

C. User Survey Questionnaire

• Sample Questions for Students:

1. How often do you visit the college library? (Daily / Weekly / Monthly / Rarely)
2. On a scale of 1 (Very Fast) to 5 (Very Slow), how would you rate the current book checkout process?
3. Have you ever been unable to find a book that the system claimed was "Available"? (Yes / No)
4. How do you currently find new books to read for your courses or interests? (Browse shelves / Ask friends / Ask librarian / Other)
5. Would you find a "Recommended for You" feature on the library portal useful? (Yes / No / Maybe)
6. How do you currently pay library fines? (Cash at desk / Other)
7. Would you prefer to pay library fines online via a portal? (Yes / No)

• Sample Questions for Library Staff:

1. On a scale of 1 (Not at all) to 5 (Very), how much of your day is spent on manual tasks like checking books in/out and managing fines?
2. How accurate is your current inventory system?
3. Do you feel you have good data on which books are most popular or which subjects are in high demand? (Yes / No)
4. Would an automated system that frees you up for research assistance and collection management be beneficial? (Yes / No)

D. Sample E-Receipt Proforma

Automated Library Assistant	
E-Receipt	
Transaction Details	
Transaction ID: 340	
Issue Date: 09 Nov 2025, 10:34 PM	
Issued To	
Name: Sarbjot Singh	
User ID: 21BCS002	
Book Details	
Title: Introduction to Algorithms	
Author: Thomas H. Cormen	
Book ID: 9780262033848	
Important Dates	
Due Date: 23 Nov 2025	
Note: Low risk of being late (37%)	
Thank you for using the Automated Library Assistant!	